

Advanced Lesson 2 - Exercises: Exact and Approximate Arithmetic

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

```
Clear[a, b, c, d, i, n, f, g, h];
```

1: How many digits of accuracy does each expression carry?

$$\left\{ \frac{3}{7}, \text{Precision}\left[\frac{3}{7}\right] \right\}$$

$$\left\{ 2 + \frac{3}{7}, \text{Precision}\left[2 + \frac{3}{7}\right] \right\}$$

$$\left\{ \text{InputForm}\left[2.\text{' + } \frac{3.\text{'}}{7.\text{'}}\right], \text{Precision}\left[2.\text{' + } \frac{3.\text{'}}{7.\text{'}}\right] \right\}$$

$$\left\{ \text{InputForm}\left[1.23\text{' + } \frac{1.2345\text{'}}{1.234567\text{'}}\right], \text{Precision}\left[1.23\text{' + } \frac{1.2345\text{'}}{1.234567\text{'}}\right] \right\}$$

$$\left\{ \text{InputForm}\left[1.2\text{' + } 1.234567890123456789\text{' 19.0915149772127 + } 1.2345678901234567890123456789\text{' 29.091514977212704}\right], \right. \\ \left. \text{Precision}\left[1.2\text{' + } 1.234567890123456789\text{' 19.0915149772127 + } 1.2345678901234567890123456789\text{' 29.091514977212704}\right] \right\}$$

$$\left\{ \text{InputForm}\left[1.234567890123456789\text{' 19.0915149772127 + } 1.2345678901234567890123456789\text{' 29.091514977212704}\right], \right. \\ \left. \text{Precision}\left[1.234567890123456789\text{' 19.0915149772127 + } 1.2345678901234567890123456789\text{' 29.091514977212704}\right] \right\}$$

$$\left\{ \text{InputForm}\left[N\left[\frac{2}{7}\right]\right], \text{Precision}\left[N\left[\frac{2}{7}\right]\right] \right\}$$

$$\left\{ N\left[\frac{2}{7}, 100\right], \text{Precision}\left[N\left[\frac{2}{7}, 100\right]\right] \right\}$$

$$\left\{ \sqrt{2}, N\left[\sqrt{2}\right] \right\}$$

Reading the results. The first two expressions are exact rationals: $3/7$ and $17/7$. Precision reports Infinity, because an exact rational has no error at all - it is not an approximation to anything.

The third and fourth are machine-precision reals. Precision reports MachinePrecision (about 15.95 decimal digits), which is what the hardware floating-point format carries. InputForm is the only way to see all the digits; the default output form politely rounds.

The fifth is the interesting one. Adding a machine number to two arbitrary-precision numbers drags everything down to machine precision - the result is only as good as its worst input. The sixth, with the 1.2 removed, keeps arbitrary precision, and Mathematica tracks how many digits survived the addition rather than pretending all of them did.

$N[2/7]$ is machine precision. $N[2/7, 100]$ is arbitrary precision, and Precision reports 100.

$\text{Sqrt}[2]$ stays as $\text{Sqrt}[2]$ because it is exact and irrational: converting it to 1.4142135623730951 would throw away infinitely many digits, so Mathematica refuses to do that unless you ask with N. This is the central discipline of the whole lesson - exact stays exact until you say otherwise.

2: Six ways to sum $1/n$ from 1 to 100

$$\sum_{n=1}^{100} \frac{1}{n}$$

$$\sum_{n=1}^{100} \frac{1.}{n}$$

$$\sum_{n=1}^{100} N\left[\frac{1}{n}\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1}{n}\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1}{n}, 50\right]$$

$$N\left[\sum_{n=1}^{100} \frac{1.}{n}, 50\right]$$

Part i returns the exact rational: a fraction with a 218-digit denominator, the 100th harmonic number, with no error whatsoever.

Parts ii and iii both return the same machine-precision real, about 5.187377517639621. Introducing 1.0 - or applying N to each term - forces every addition into floating point.

Part iv returns the same machine real, but by a better route: the sum is performed exactly and only the final answer is approximated. No rounding error accumulates across the hundred additions.

Part v returns fifty correct digits, for exactly that reason: the exact sum is still available, so N can be asked for as many digits as wanted.

Part vi fails to give fifty digits. The 1.0 destroyed the exact sum before N ever saw it. All that reaches N is a machine number carrying sixteen digits; N cannot manufacture the missing thirty-four. Precision is lost at the point where an inexact number enters the calculation, and no later request can recover it.

3: E^{π} to 500 digits, and exactly

`N[eπ, 500]`

e^{π}

E^{π} is exact and transcendental. There is no finite exact form simpler than E^{π} itself, so evaluating it "exactly" returns it unchanged - Mathematica has nothing better to offer. Ask for 500 digits and it computes them, on demand, from the exact form. Note that E and Pi must be the built-in constants, not the plain symbols e and p, or the expression means nothing.

4: A complex sum

$$\sum_{n=1}^{10} \frac{1}{n + 2i}$$

$$N\left[\sum_{n=1}^{10} \frac{1}{n + 2i}, 50\right]$$

$$N\left[\sum_{n=1}^{10} \frac{1}{n + 2.\,i}, 50\right]$$

The exact sum is a Gaussian rational: one exact rational for the real part and one for the imaginary part. Asking for fifty digits works, for the same reason as Exercise 2 part v - the exact value is still there to approximate. Writing 2.0 I instead of 2 I makes every term a machine complex number, the exact value is gone before N is called, and only about sixteen digits come back. Same failure, same cause.

5: Sin[6] exactly and approximately

```
Sin[6]
```

```
Sin[6.`]
```

```
N[Sin[6], 50]
```

```
N[Sin[6.`], 50]
```

```
N[Sin[N[6, 20]], 50]
```

Sin[6] stays unevaluated: 6 radians is not a special angle, so there is no exact closed form and Mathematica keeps the exact expression. Sin[6.0] returns a machine real, about -0.279415.

N[Sin[6], 50] gives fifty correct digits, computed from the exact Sin[6].

N[Sin[6.0], 50] does not. The argument was already a machine number, so Sin[6.0] evaluated to sixteen good digits before N was reached, and the request for fifty arrives too late.

N[Sin[N[6, 20]], 50] does not give fifty digits either, but for a sharper reason: the argument was deliberately built with only twenty digits of precision, so the sine of it is only known to fewer than twenty digits - Mathematica reports about 18.7. The outer request for fifty cannot exceed what the input supports, and taking a sine costs a little precision on top. Mathematica tracks this automatically and reports the honest precision rather than padding with noise.

6: The partial sums as a user function

```
Clear[f, n, nTerms];
```

```
f[nTerms_] :=  $\sum_{n=1}^{nTerms} \frac{1}{n}$ 
```

```
{f[1], f[10], f[100]}
```

Delayed assignment is what keeps this exact. Every value returned is a rational: 1, then 7381/2520, then the 218-digit harmonic fraction.

7: How many system functions are there?

Names["System`*"] returns the whole list of built-in names. It is far too long to print in a submitted notebook, so only its length is evaluated below; the list itself is computed and discarded.

```
Length[Names["System`*"]]
```

The backquote is a context mark, not an apostrophe. `System`` is the context holding every built-in symbol, and `Global`` is the context your own symbols land in - which is the subject of Lesson 15.

8: myDiff1 and myDiff2 - why differentiation needs delayed assignment

```
Clear[x, f];
myDiff1[f_] =  $\partial_x f$ 
{myDiff1[x2 + 3 x + 4], myDiff1[e2 x + 3], myDiff1[f[x]]}
?myDiff1
```

Part 2, and why it fails. With immediate assignment the right-hand side is evaluated once, at definition time. At that moment `f` is just an unassigned symbol, and the derivative of a symbol that does not involve `x` is zero. So `D[f, x]` evaluates to 0, and 0 is what gets stored. `"?myDiff1"` confirms it: the stored rule is `myDiff1[f_] := 0`. Every call returns 0, whatever you hand it. The definition never had a chance to see a real argument.

```
Clear[x, f];
myDiff2[f_] :=  $\partial_x f$ 
{myDiff2[x2 + 3 x + 4], myDiff2[e2 x + 3], myDiff2[f[x]]}
?myDiff2
```

Part 4, and why it works. Delayed assignment stores `D[f, x]` literally, unevaluated. The differentiation is postponed until the function is actually called, by which time the pattern variable `f` has been bound to a real expression. The three calls return `3 + 2 x`, then `2 E^(2 x)`, and then `f'[x]` - Mathematica correctly reports the derivative of an unknown function as its derivative rather than giving up.

Part 5. Delayed assignment is mandatory here because the whole point of the function is to operate on an argument that does not exist yet. Any definition whose right-hand side is meaningless until the argument arrives - differentiation, integration, random generation, anything that must be recomputed per call - has to use `:=`. Immediate assignment is only correct when the right-hand side can be fully computed once and reused forever.